esprit
Se former autrement

SAFEPAY

SECURE DIGITAL PAYMENT PLATFORM

Conception d’une architecture microservices

Encadré Par : Ben Salah Ons

Année universitaire 2026-2027

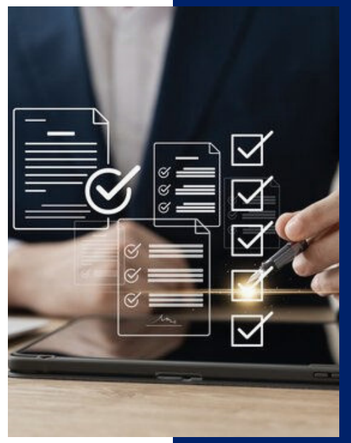
Notre équipe

Yassmine Rahmouni

Yosr kobbi

Mohamed Aziz Bougacha

Fedi Mhiry



Contexte & problématique 01

Acteurs & fonctionnalités 04

Diagramme de cas d’utlisation 11

Architecture globale proposée 13

Découpage métier & propriété des données 15

La conception des microservices 17

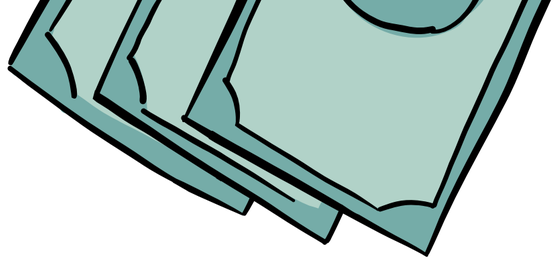
Diagramme de séquence 23

Choix technologiques & sécurité 25

Organisation d’équipe & dépôt Git 15

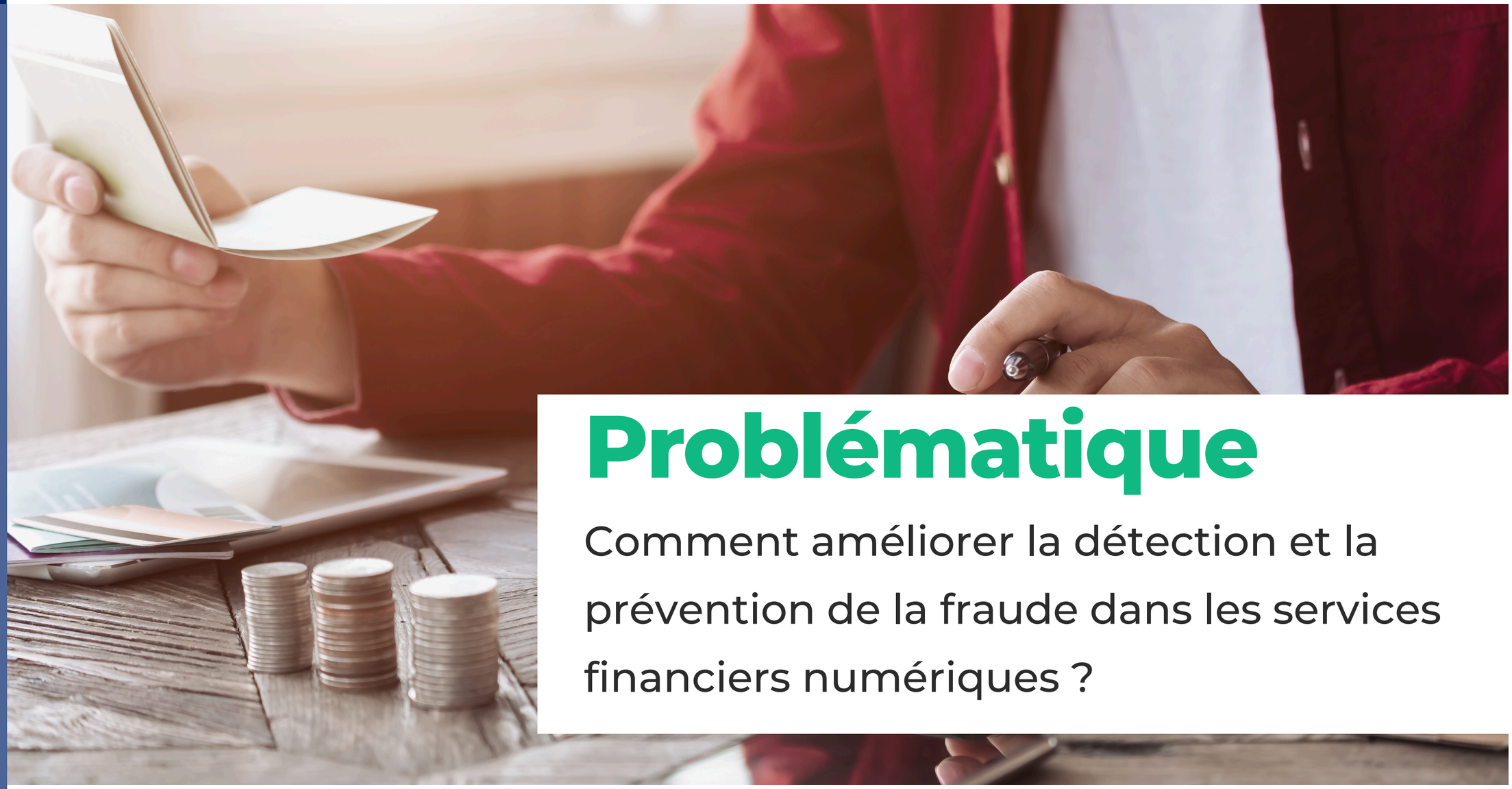
Planning, risques et conclusion 15

Contexte & problématique



Contexte

- Paiements et transferts numériques en forte progression
- Risques de fraude, phishing et transactions non autorisées
- Détection tardive des opérations anormales
- Besoin de contrôler le risque avant le mouvement des fonds

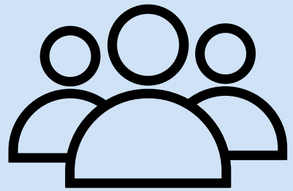


Problématique

Comment améliorer la détection et la prévention de la fraude dans les services financiers numériques ?

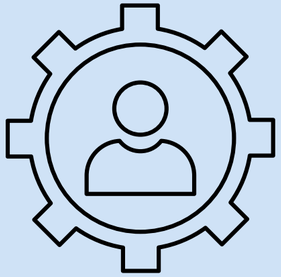
Acteurs & fonctionnalités

Identification des acteurs



Client (humain)

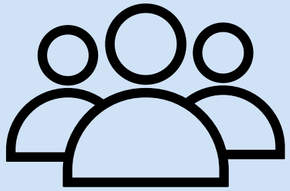
analyste fraude (humain)



Admin (humain)

Fraud Detection (acteur système)

Identification des acteurs



Utilisateur final de la plateforme. Il gère son compte, son portefeuille et ses cartes virtuelles, et effectue des paiements et des transferts.

Client

analyste fraude (humain)

Admin (humain)

Fraud Detection (acteur système)

Identification des acteurs

analyste fraude

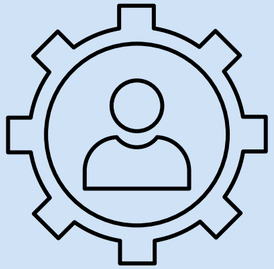
Responsable de la supervision. Il examine les transactions signalées comme suspectes et consulte les alertes..

Client (humain)

Admin (humain)

Fraud Detection (acteur système)

Identification des acteurs



Responsable de la gestion des comptes. Il gere les utilisateurs (suspension, réactivation, decision KYC) et consulte les alertes.

Admin

Client (humain)

analyste fraude (humain)

Fraud Detection (acteur système)

Identification des acteurs

Fraud Detection (acteur système)

Acteur automatique. Il évalue chaque transaction en temps reel et leve une alerte lorsqu'elle est jugée suspecte.

Client (humain)

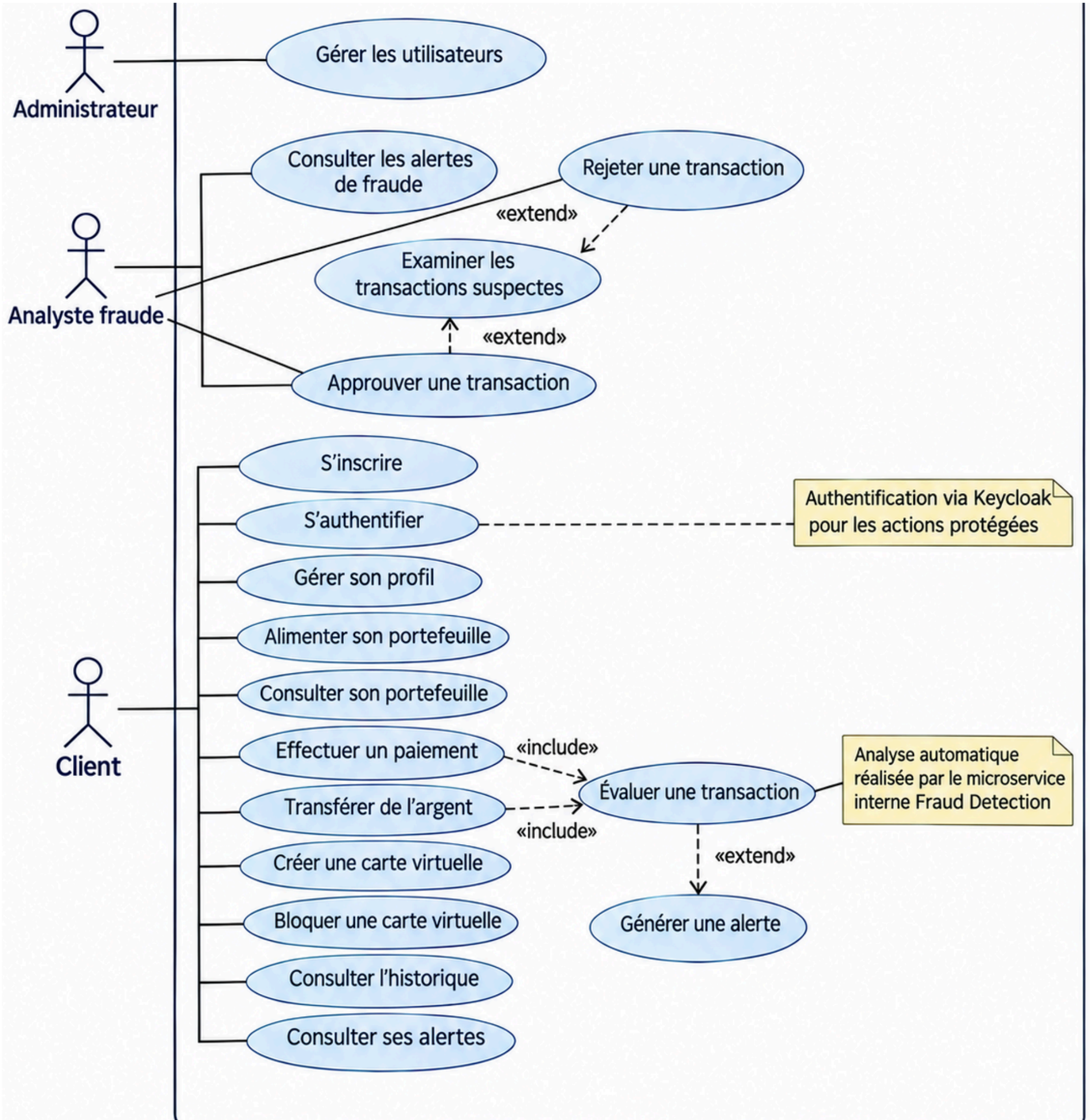
analyste fraude (humain)

Admin (humain)

Les principales fonctionnalités

- Paiement et transfert d’argent
- Historique et traçabilité des opérations
- Alertes et décision de l’analyste
- Gestion des soldes et cartes virtuelles
- Scoring antifraude avant exécution

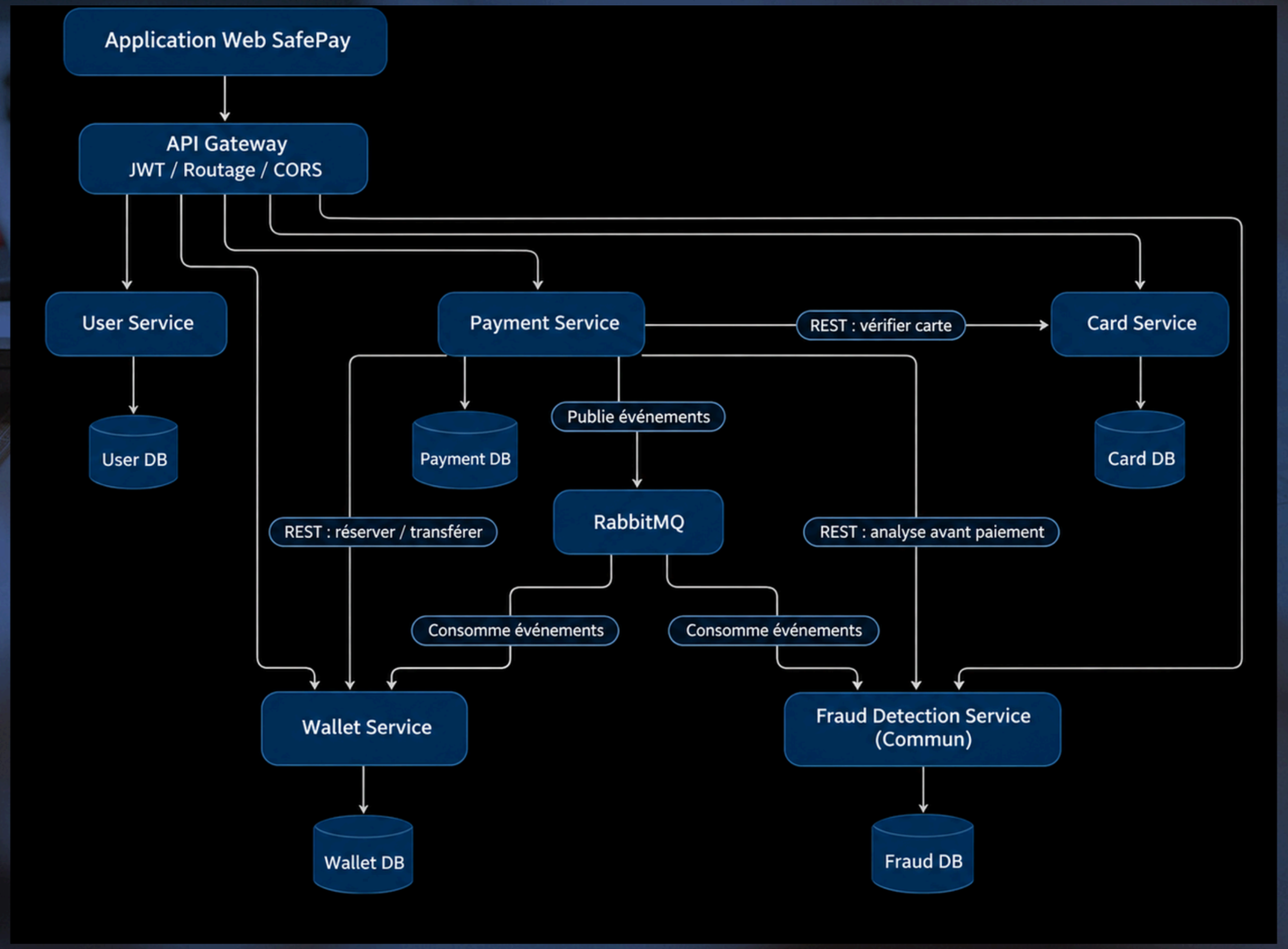
Diagramme de cas d’utlisation


Administrateur
Gérer les utilisateurs
Analyste fraude
Consulter les alertes de fraude
Rejeter une transaction
«extend»
Examiner les transactions suspectes
«extend»
Approuver une transaction
Client
S'inscrire
S'authentifier
Authentification via Keycloak pour les actions protégées
Gérer son profil
Alimenter son portefeuille
Consulter son portefeuille
Effectuer un paiement
«include»
Évaluer une transaction
Analyse automatique réalisée par le microservice interne Fraud Detection
Transférer de l'argent
«include»
«extend»
Créer une carte virtuelle
Générer une alerte
Bloquer une carte virtuelle
Consulter l'historique
Consulter ses alertes

SafePay TN

L’architecture globale proposée

Diagramme global



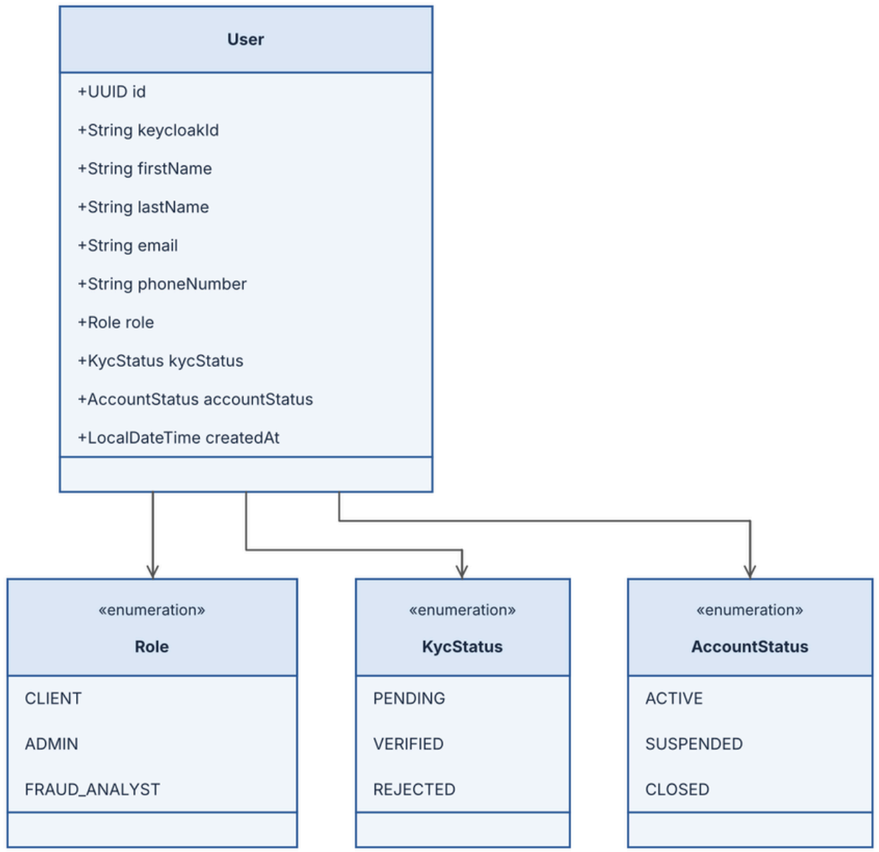
Découpage métier & propriété des données

Microservice	Base de données	Responsabilité	Entités principales
MS1 User Service	H2	Profil, KYC, rôles	users
MS2 Wallet Service	H2	Soldes, crédits et débits	wallets , wallet_transactions
MS3 Payment Service	H2	Paiements, statuts, orchestration	payments + status_history
MS4 Card Service	MySQL	Cartes virtuelles, plafonds	cards + status_history
MS5 Fraud Service	PostgreSQL	Scoring, alertes, décisions	checks + alerts

La conception des microservices

MS1 – User Service

Diagramme de classe



API REST • CONTRATS

GET	/api/users/me	200
PATCH	/api/users/me	200
GET	/internal/users/{id}/eligibility	200
PATCH	/api/admin/users/{id}/status	200
PATCH	/api/admin/users/{id}/kyc	200

INTERACTIONS

SYNCHRONE • REST

Wallet / Card / Payment interrogent l'éligibilité (GET REST).

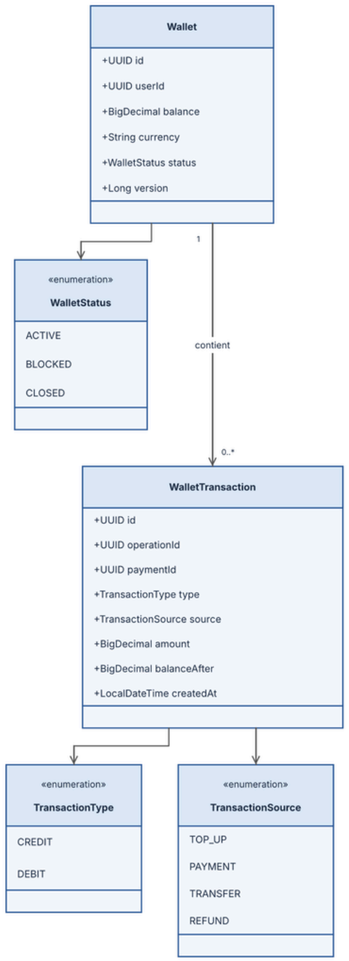
ASYNCHRONE • RabbitMQ

Publie user.suspended et user.reactivated.

Codes usuels : 400 requête invalide • 401 non authentifié • 403 interdit • 404 introuvable

MS2 – Wallet Service

Diagramme de classe



API REST • CONTRATS

POST	/api/wallets	201
GET	/api/wallets/me	200
GET	/api/wallets/{id}/balance	200
GET	/api/wallets/{id}/transactions	200
POST	/internal/wallets/transfers	200
GET	/internal/wallets/{id}/eligibility	200

INTERACTIONS

SYNCHRONE • REST

Payment appelle Wallet pour débiter/créditer atomiquement.

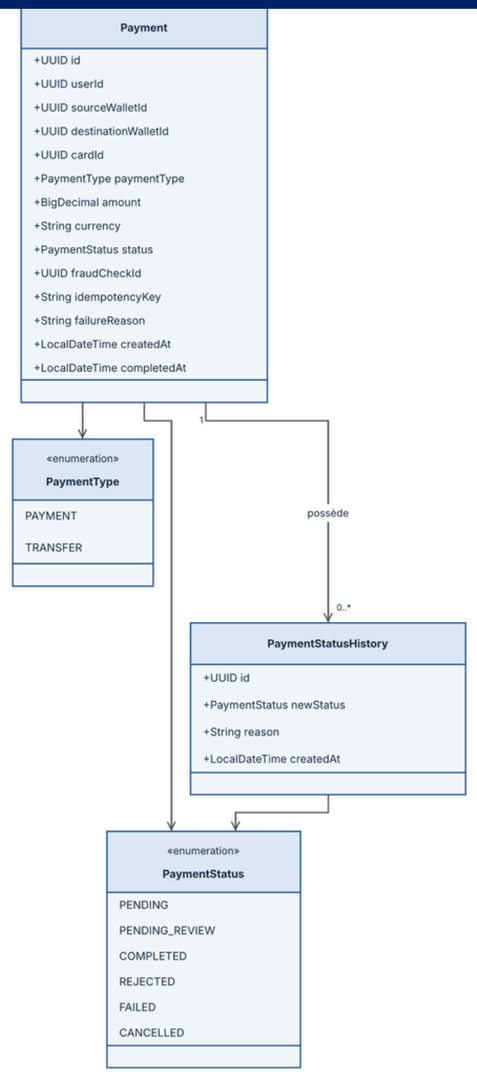
ASYNCHRONE • RabbitMQ

Publie wallet.transfer.completed ; consomme user.suspended.

Codes usuels : 400 requête invalide • 401 non authentifié • 403 interdit • 404 introuvable

MS3 – Payment Service

Diagramme de classe



API REST • CONTRATS

POST	/api/payments	202
POST	/api/transfers	202
GET	/api/payments	200
GET	/api/payments/{id}	200
POST	/api/payments/{id}/cancel	200
GET	/api/payments/{id}/receipt	200

INTERACTIONS

SYNCHRONE • REST

REST vers User, Fraud, Card et Wallet avant le transfert.

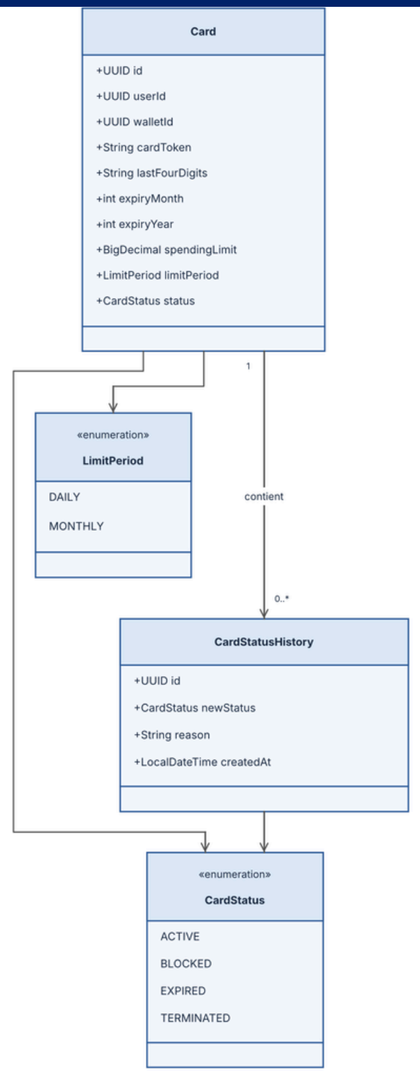
ASYNCHRONE • RabbitMQ

Publie payment.completed / failed ; consomme fraud.review.completed.

Codes usuels : 400 requête invalide • 401 non authentifié • 403 interdit • 404 introuvable

MS4 – Card Service

Diagramme de classe



API REST • CONTRATS

POST	/api/cards	201
GET	/api/cards	200
GET	/api/cards/{id}	200
PATCH	/api/cards/{id}/block	200
PATCH	/api/cards/{id}/unblock	200
PUT	/api/cards/{id}/limit	200
POST	/internal/cards/{id}/validate-payment	200

INTERACTIONS

SYNCHRONE • REST

Payment demande si la carte est active et autorisée.

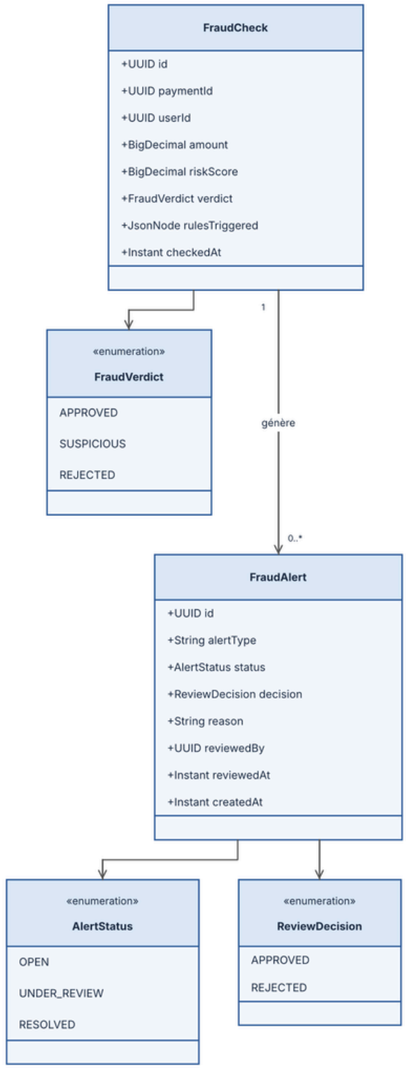
ASYNCHRONE • RabbitMQ

Publie card.blocked ; consomme user.suspended.

Codes usuels : 400 requête invalide • 401 non authentifié • 403 interdit • 404 introuvable

MS5 – Fraud Detection Service

Diagramme de classe



API REST • CONTRATS

POST	/internal/fraud/checks	200
GET	/api/admin/fraud/checks/{id}	200
GET	/api/admin/fraud/alerts	200
GET	/api/admin/fraud/alerts/{id}	200
PATCH	/api/admin/fraud/alerts/{id}/decision	200

INTERACTIONS

SYNCHRONE • REST

Payment obtient un verdict APPROVED / SUSPICIOUS / REJECTED.

ASYNCHRONE • RabbitMQ

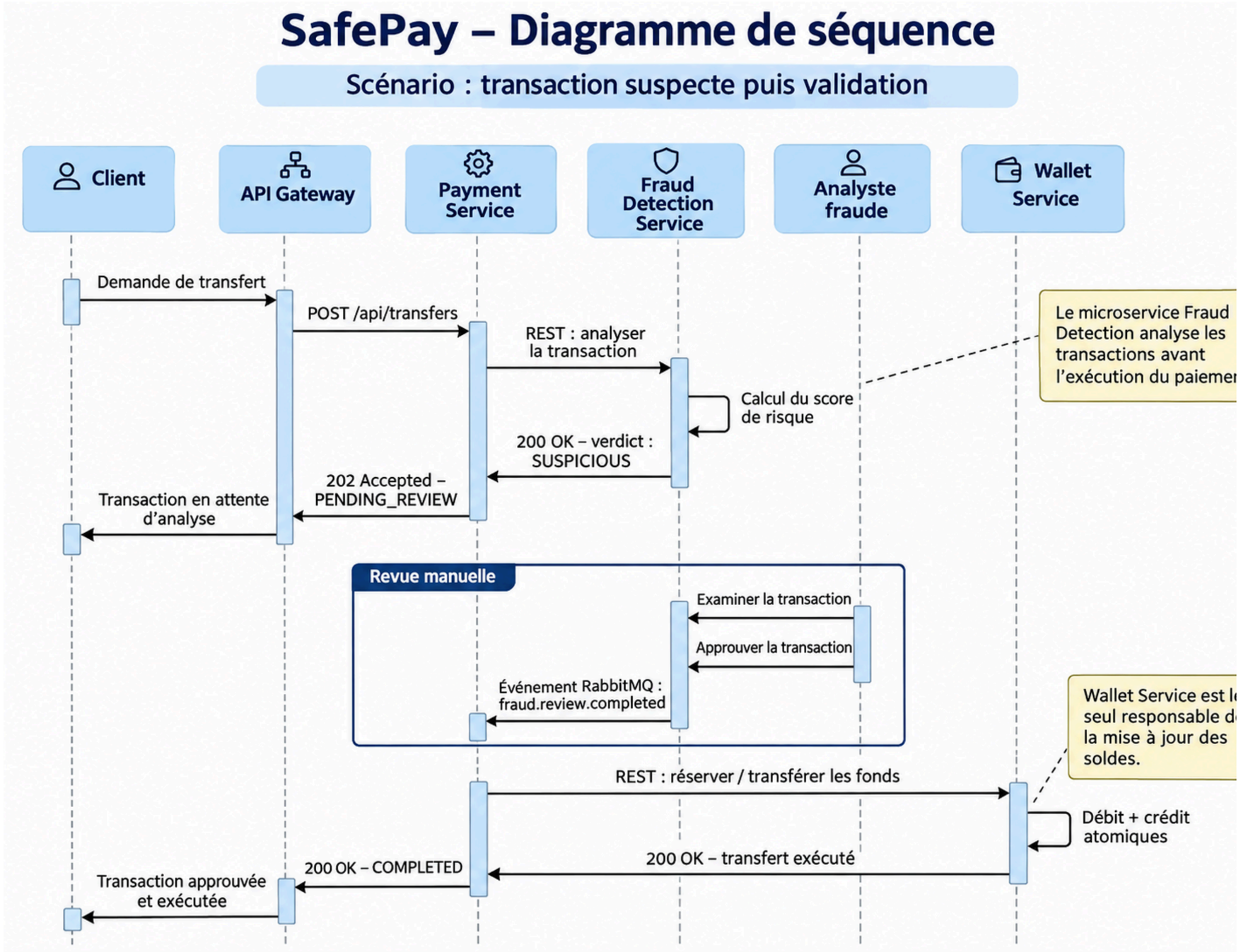
Publie fraud.alert.created et fraud.review.completed.

Codes usuels : 400 requête invalide • 401 non authentifié • 403 interdit • 404 introuvable

Diagramme de séquence

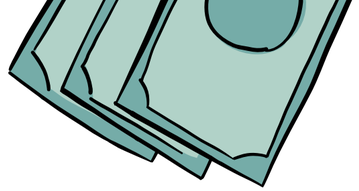
SafePay – Diagramme de séquence

Scénario : transaction suspecte puis validation



Choix technologiques & sécurité

Choix technologiques & sécurité



Backend

Spring Boot, Java et Spring Data JPA.

Frontend

Angular.

Communication

REST, OpenFeign et RabbitMQ.

Sécurité

Keycloak, OAuth2/OIDC et JWT.

Découverte services

Eureka et Config Server.

Données

H2 (pour 3 services), MySQL et PostgreSQL.

Déploiement

Docker et Docker Compose.

API et résilience

Swagger/OpenAPI, timeouts et idempotence.

Organisation d’équipe & dépôt Git

Organisation d’équipe

User Service
Rahmouni Yassmine

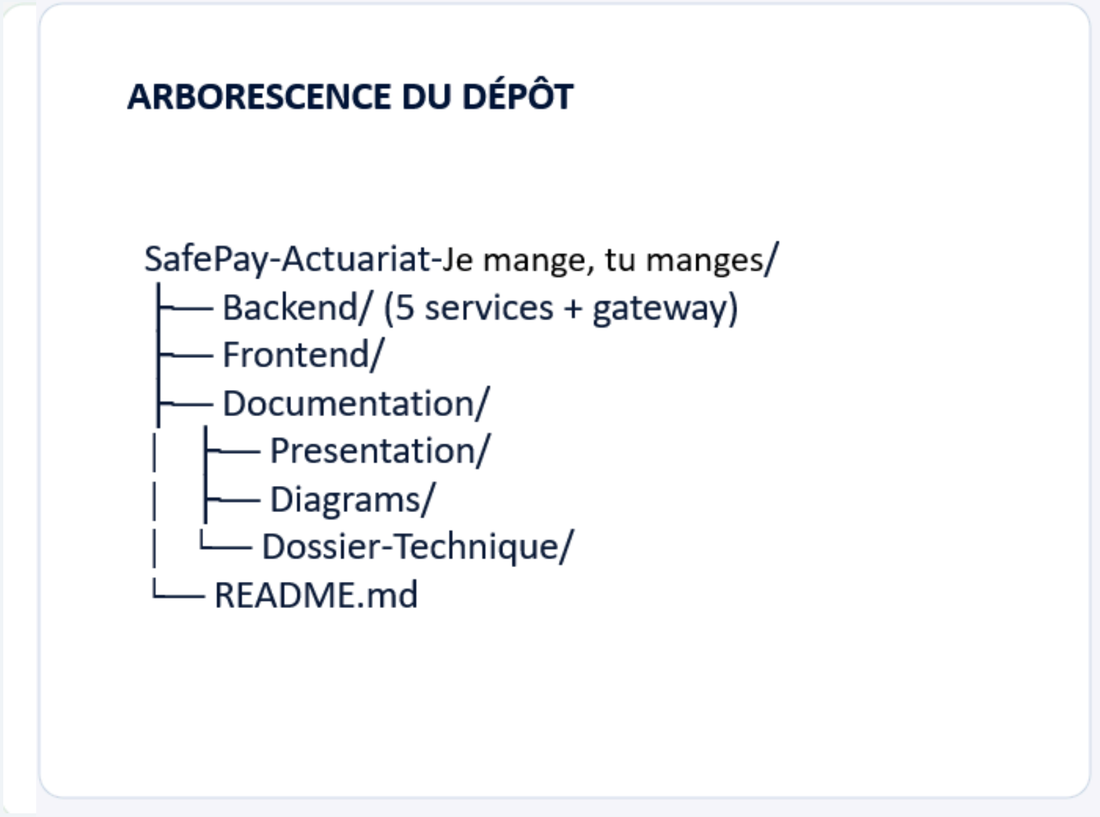
Wallet Service
Bougacha Mohamed Aziz

Payment Service
Kobbi Yosr

Card Service
Mhiri Fedi

Fraud Detection
Toute l’équipe

ARBORESCENCE DU DÉPÔT



Planning, risques & conclusion

PLANNING PRÉVISIONNEL

S1

Analyse, besoins et UML

S2

Bases de données et contrats API

S3-4

Développement des services

S5

REST + RabbitMQ + frontend

S6

Tests, intégration, documentation

RISQUES & MESURES

Risque	Mesure
Contrats incompatibles	OpenAPI partagé
Double débit	Idempotence + atomicité
Fraud indisponible	Refus par défaut
Événements dupliqués	Consumers idempotents
Retard d'intégration	Jalons + Git

Conclusion

SafePay TN
application mobile anti-fraude

En conclusion, SafePay repose sur cinq microservices indépendants qui facilitent la maintenance et l'évolutivité de la plateforme. Grâce à REST, RabbitMQ et Keycloak, cette architecture permet une communication efficace et sécurisée, tandis que Fraud Detection contribue à prévenir les transactions frauduleuses.

MERCI DE VOTRE ATTENTION